

Algorithms Notes — Skiena (TADM 3e) + CLRS (4e), with C++ from Weiss (4e)

Start at [INDEX.md](#) — module map, CLRS↔Skiena crosswalk, study order, and the C++ conventions used throughout.

What is in this archive

16 markdown files: `INDEX.md` plus `M01–M15`. Every module is a unified synthesis of Skiena and CLRS on one topic, with all implementation code in **C++17**.

State of the refinement pass (as of this archive)

A refinement pass is underway. It has three parts; two are complete across **all** files, the third is complete for **M01–M08**.

Done everywhere (INDEX + M01–M15)

using 647 `std::` prefixes removed from code blocks. Every block assumes the
namespace prelude `#include <bits/stdc++.h> + using namespace std;`
std;

everywhere

Every code block compiles All 16 files verified under `g++ -std=c++17 -Wall -Wextra`. Illustrative fragments were rewritten into real compilable functions rather than left as snippets.

All links resolve 284 links checked, **0 broken**. Fixed a genuinely broken filename (`M13-graph-traversal.md` → `M13-graphs-traversal.md`) and repointed 17 forward-references to modules that do not exist yet at the INDEX roadmap.

INDEX rewritten Module map now carries a status column ( written /  planned), plus a new "C++ conventions" section.

Done for M01–M08 only

Each of these modules now ends with three new sections:

1. **Practice — where to drill this module.** A table of specific problems (LeetCode number + title + link), plus CSES and Codeforces tag pages. **Every LeetCode slug was verified against live search**, not written from memory.
2. **C++ Toolkit for This Module.** A primer on the C++ features that module's code leans on, grounded in Mark Allen Weiss, *Data Structures and Algorithm Analysis in C++*, 4th ed., ch. 1 — parameter passing, return passing, references, the Big-Five, templates, function objects, iterator invalidation, and so on, cited by section and page.
3. **Appendix — C++ for Every Pseudocode Block.** A heavily-commented, runnable C++ translation of *every* pseudocode block in that module, kept line-for-line faithful to the pseudocode (1-based indexing and all), so you can read them side by side. **Each pseudocode block in the body carries a `→ C++ implementation:` link straight to its appendix entry.**

Appendix coverage so far: **61 pseudocode blocks** implemented and linked.

MODULE	PSEUDOCODE BLOCKS LINKED
M01 Foundations	6
M02 Asymptotics	6
M03 Divide & Conquer	5
M04 Randomization	5
M05 Sorting	10
M06 Elementary Data Structures	3
M07 Hashing	9
M08 Search Trees	17

Not yet done

M09–M15 (Amortized, Union-Find, Dynamic Programming, Greedy, Graph Traversal, MST, Shortest Paths) have the `std::` cleanup, the compile verification and the link fixes, but **not yet** the Practice / C++ Toolkit / Appendix sections. Their pseudocode blocks do not yet carry `→ C++ implementation:` links. They already contain substantial C++ implementations in the body — those are unchanged and correct.

Body code vs appendix code

They are deliberately different, and comparing them is the point:

- **Body implementations** are the *practical* version — what you would actually write, 0-indexed, using the STL where it helps.
 - **Appendix implementations** are *literal translations of the pseudocode*, line for line, keeping CLRS's 1-based indexing and variable names, with comments explaining both the algorithm and the C++.
-

Sources

BOOK	USED FOR
Steven S. Skiena, <i>The Algorithm Design Manual</i> , 3rd ed.	intuition, modelling, pattern recognition, war stories, implementation pragmatics
Cormen, Leiserson, Rivest, Stein, <i>Introduction to Algorithms</i> , 4th ed.	formal definitions, invariants, proofs, recurrences, rigorous complexity
Mark Allen Weiss, <i>Data Structures and Algorithm Analysis in C++</i> , 4th ed.	the C++ language teaching only — cited as [Weiss §1.5.3, p.25]

Citations look like [CLRS §2.1, p.17] and [Skiena §1.3, p.11] so you can go back to the source quickly.